

Vulkan Notes

Window Surface, Swap Chain & Image Views

Adam HAMOUCH

June 2026

1. Window Surface

Vulkan est agnostique de la plateforme et ne peut pas interagir directement avec le gestionnaire de fenetres. La **window surface** (`VkSurfaceKHR`) est l'interface abstraite entre Vulkan et la fenetre GLFW — c'est sur elle qu'on effectue les rendus.

Elle fait partie de l'extension WSI (`VK_KHR_surface`), deja incluse dans les extensions retournees par `glfwGetRequiredInstanceExtensions`. La creation de la surface est **specifique a la plateforme** en bas niveau (HWND sur Windows, XCB sur Linux), mais GLFW l'abstrait avec `glfwCreateWindowSurface`.

La surface est **optionnelle** : pas necessaire pour le rendu off-screen ou le compute pur.

Ordre dans `initVulkan` : la surface doit etre creee apres l'instance mais **avant** le physical device, car elle influence le choix du GPU (certains GPU ne supportent pas la presentation sur une surface donnee).

Queue present : la presentation necessite une queue family qui supporte la presentation sur cette surface. Graphics et present peuvent etre la meme famille ou des familles differentes selon le GPU. On utilise un `std::set` pour eviter de creer deux fois la meme queue.

Destruction : `vkDestroySurfaceKHR` avant `vkDestroyInstance`.

2. Swap Chain

Vulkan n'a pas de framebuffer par default. La **swap chain** est la file d'attente d'images que le GPU rend et qui sont presentees a l'ecran. Pipeline : acquerir une image libre → rendre dedans → la retourner a la file → ainsi de suite.

Ce n'est pas une fonctionnalite core de Vulkan — c'est l'extension `VK_KHR_swapchain` a verifier et activer sur le logical device.

Trois proprietes a verifier sur le GPU

Propriete	Ce qu'elle definit
Capabilities	Nombre min/max d'images, taille min/max
Formats	Format pixel (ex. <code>VK_FORMAT_B8G8R8A8_SRGB</code>) + color space
Present modes	Conditions d'affichage des images

On stocke ces infos dans une struct `SwapChainSupportDetails` et on choisit les meilleurs parametres disponibles.

Les modes de presentation

- **IMMEDIATE** : image affichee immediatement, tearing possible.
- **FIFO** : file d'attente strict, GPU bloque si pleine. Seul mode garanti par le standard. Equivalent du VSync.
- **FIFO_RELAXED** : comme FIFO mais affichage immediat si la file etait vide.
- **MAILBOX** : prefere. Si la file est pleine, la nouvelle frame remplace la plus recente en attente → triple buffering, pas de tearing, GPU jamais bloque, latence minimale. Fallback sur FIFO si indisponible.

Parametres de creation

- **Nombre d'images** : `minImageCount + 1` pour eviter d'attendre que le driver libere une image.
- **Extent** : resolution en pixels via `glfwGetFramebufferSize` (pas `glfwGetWindowSize` — different sur les ecrans HiDPI/Retina).
- **Sharing mode** : **EXCLUSIVE** si graphics et present sont la meme famille (plus performant), **CONCURRENT** sinon.
- **Pre-transform** : transformation a appliquer aux images (rotation...). En general **currentTransform** pour ne rien faire.
- **Composite alpha** : gestion de la transparence avec le gestionnaire de fenetres. En general **OPAQUE**.
- **Clipped** : **VK_TRUE** pour ignorer les pixels obscurcis par d'autres fenetres → meilleures performances.

Destruction : `vkDestroySwapchainKHR` avant `vkDestroySurfaceKHR`. Les `VkImage` de la swap chain sont detruites automatiquement avec elle — pas besoin de les detruire manuellement.

3. Image Views

En Vulkan, on n'accede jamais directement a une `VkImage` — on passe toujours par une `VkImageView`.

VkImage = memoire GPU brute, juste des octets.

VkImageView = mode d'emploi pour interpreter cette memoire : type (2D, cubemap...), format, canaux a exposer, niveaux de mipmap.

Analogie : `VkImage` = fichier .raw, `VkImageView` = logiciel qui sait comment l'ouvrir.

On cree une **image view par image** de la swap chain. Les parametres qui varient selon le cas :

- **viewType** : 2D pour swap chain et textures, **CUBE** pour les cubemaps (skybox).
- **format** : `swapChainImageFormat` pour la swap chain, `R8G8B8A8_SRGB` pour les textures, `D32_SFLOAT` pour le depth buffer.
- **aspectMask** : `COLOR_BIT` pour la couleur, `DEPTH_BIT` pour le depth buffer.
- **levelCount** : 1 pour la swap chain, `mipLevels` pour les textures avec mipmaps.
- **layerCount** : 1 en general, 6 pour un cubemap.

Dans un moteur, on abstrait ca dans une fonction helper `createImageView(image, format, aspect, mipLevels)`.

Destruction : les image views doivent etre detruites **manuellement** avec `vkDestroyImageView` — contrairement aux `VkImage` de la swap chain, elles ne sont pas detruites automatiquement.

Ordre de creation / destruction

Creation : Instance → Surface → Physical Device → Logical Device → Swap Chain → Image Views

Destruction : Image Views → Swap Chain → Surface → Instance

Toujours detruire dans l'ordre inverse de creation.